

NexusOS Service Standard v1

Version: 1.0

Status:

-  Approved
-  Frozen
-  Active Standard

Purpose

Models describe data.

Services make business decisions.

This document defines the mandatory architecture for every Service in NexusOS before implementing business logic.

Core philosophy:

Models represent state. Services execute use cases.

1. One Use Case per Class

Each Service class represents exactly one business action.

Example:

```
app/Services/TenantModule/  
  EnableModuleForTenant.php  
  DisableModuleForTenant.php  
  OverrideModuleForTenant.php  
  SyncTenantModulesFromPlan.php
```

Avoid large service classes such as:

```
TenantModuleService
```

containing multiple unrelated methods.

2. Group by Domain

Services are grouped by Domain.

Never create a flat Services directory.

Example:

```
app/Services/  
  
TenantModule/  
TenantLicense/  
TenantUser/  
Role/  
Permission/
```

3. Public Execution Method

Every Service exposes exactly one public method:

```
handle()
```

Never use:

```
__invoke()
```

Reason:

Explicit over Magic.

4. Constructor Responsibility

Constructor contains only injected dependencies.

Example:

```
public function __construct(  
    private readonly AuditLogger $auditLogger,  
) {}
```

Never pass runtime values through the constructor.

5. handle() Responsibility

Runtime input belongs only to:

```
handle(...)
```

Example:

```
handle(  
    string $tenantId,  
    string $moduleId,  
    ?string $actorUserId = null,  
)
```

Use primitive values and IDs.

Do not pass hydrated Eloquent Models.

The Service performs a fresh database load inside its own transaction.

Benefits:

- Queue-safe
 - Job-safe
 - No stale models
 - No hidden caller assumptions
-

6. Transactions

Every write Service owns its transaction.

Pattern:

```
return DB::transaction(function () {  
    ...  
});
```

Read-only Services normally do not require transactions.

7. Audit Consistency

Every domain write and its audit record must occur inside the same transaction.

Never create audit records later.

The domain change and audit entry must succeed or fail together.

8. Bulk / Sync Operations

Bulk operations own one transaction around the entire operation.

Never call write-heavy child Services repeatedly inside loops.

Instead:

- one transaction
- direct writes
- shared internal helper methods if needed

Example:

```
SyncTenantModulesFromPlan
```

should not call:

```
EnableModuleForTenant
```

once per module.

9. Dependency Injection

Constructor:

- Dependencies only

handle():

- Runtime input only

Services are resolved by the Laravel Container.

10. Return Values

Write Services return DTOs.

Naming:

```
{UseCaseName}Result
```

Examples:

```
EnableModuleForTenantResult  
DisableModuleForTenantResult
```

Location:

```
app/Data/{Domain}/
```

Read-only Services may return Models or Collections only when the operation is a simple query and does not represent a business decision.

11. Validation

Validation is divided into two categories.

Input Validation

Examples:

- ULID format
- Required values
- Data shape

Performed at:

- FormRequest
 - CLI Command
 - Job
 - Guard clauses
-

Business Rule Validation

Performed inside the Service.

Examples:

- Module belongs to current plan
- Tenant is active
- Override protection
- License validity

Business rules are part of the Use Case itself.

Do not create separate Domain Validator classes by default.

12. Domain Exceptions

Business rule failures throw specific exceptions.

Never throw generic RuntimeException.

Location:

```
app/Exceptions/{Domain}/
```

Examples:

```
ModuleNotAllowedForPlanException  
  
ProtectedModuleSourceException  
  
TenantModuleAlreadyEnabledException  
  
RoleDoesNotBelongToTenantException
```

13. Authorization

Authorization is split into three layers.

Tenant Isolation

Handled by:

- Middleware
- Tenant Context

Permission Evaluation

Handled by:

- Policies
- Gates
- Permission Engine

Performed before calling the Service.

Business Rules

Handled inside the Service.

Services never call:

- Auth
- Gate
- Policy

Sensitive Services may receive explicit capability flags.

Example:

```
handle(  
    ...,  
    bool $isPlatformActor,  
)
```

14. Folder Structure

Services use singular Domain names.

Example:

```
TenantModule  
  
TenantLicense  
  
TenantUser  
  
Role
```

Never plural.

Services are classified by the primary Domain they modify.

Example:

```
SyncTenantModulesFromPlan
```

belongs to


```
TenantModule
```

because it writes:

```
tenant_modules
```

even if triggered by a license change.

Standard Directory Structure

```
app/  
  
Services/  
  TenantModule/  
  TenantLicense/  
  TenantUser/  
  Role/  
  Permission/  
  
Data/  
  TenantModule/  
  TenantLicense/  
  TenantUser/  
  Role/  
  
Exceptions/  
  TenantModule/  
  TenantLicense/  
  TenantUser/  
  Role/
```

Service Checklist

Every Service must satisfy the following rules.

- One Use Case per class.
- Grouped by Domain.
- Public method is `handle()`.
- Constructor contains dependencies only.

- Runtime input is passed to `handle()`.
 - Runtime input uses primitive IDs.
 - Write Services own their transaction.
 - Audit records are written inside the same transaction.
 - Bulk operations own one transaction.
 - Bulk operations never call write-heavy Services inside loops.
 - Dependencies are injected through the constructor.
 - Write Services return DTOs.
 - DTOs live in `app/Data/{Domain}`.
 - Input validation happens at the application edge.
 - Business rules live inside the Service.
 - Domain failures throw specific exceptions.
 - Services never call Auth, Gate or Policy.
 - Authorization is completed before Service execution.
 - Folder names use singular Domain names.
 - Services are classified by the Domain they modify.
-

First Reference Implementation

The first Service implemented under this standard is:

```
app/Services/TenantModule/EnableModuleForTenant.php
```

It serves as the reference implementation for all future NexusOS Services.